DevOps Practice Guide: Beginner to Expert

Project: DevOps Shop (Spring Boot + React + MySQL Microservices)
Budget: $200 AWS Credits
Tools: AWS Services, Jenkins, Terraform, EKS, ECS, Git, Observability

---

Phase 1: Foundations (Beginner)

Estimated AWS Cost: $0–$5

Module 1 — Git & Version Control

#	Exercise	What You'll Learn
1	Initialize repo, push to GitHub	`git init`, remotes, `.gitignore`
2	Create `develop`, `staging`, `main` branches	Branching strategy (GitFlow)
3	Feature branch → PR → merge for adding a health endpoint to product-service	Pull requests, code review flow
4	Simulate a merge conflict in `docker-compose.yml` and resolve it	Conflict resolution
5	Tag releases: `v1.0.0`, `v1.1.0`	Semantic versioning, `git tag`
6	Set up branch protection rules on `main`	GitHub settings, enforced reviews
7	Write conventional commit messages, add a `CHANGELOG.md`	Commit hygiene

Module 2 — Docker Deep Dive

#	Exercise	What You'll Learn
1	Analyze each Dockerfile — identify stages, layers, caching	Multi-stage builds
2	Optimize `product-service/Dockerfile` — reduce image size by 50%+	`.dockerignore`, layer ordering, `jre` vs `jdk`
3	Run `docker-compose up`, test all endpoints from README	Compose networking, depends_on, healthchecks
4	Add a `docker-compose.override.yml` for local dev (hot reload, debug ports)	Compose overrides
5	Use `docker scan` or `trivy` to scan all images for CVEs	Container security
6	Create a `docker-compose.prod.yml` with resource limits and restart policies	Production hardening
7	Push images to Docker Hub with tags `latest` and `v1.0.0`	Registry basics

Module 3 — Linux & Networking Basics

#	Exercise	What You'll Learn
1	Launch a free-tier EC2 (t2.micro, Amazon Linux 2023), SSH in	EC2, key pairs, security groups
2	Install Docker + Docker Compose on EC2, deploy the app manually	Linux package management
3	Use `curl`, `netstat`, `ss`, `dig` to debug service connectivity	Network troubleshooting
4	Set up `systemd` service for Docker Compose auto-start on boot	Process management
5	Configure `ufw`/`iptables` — only allow ports 22, 80, 443	Firewall basics

---

Phase 2: AWS Core Services (Beginner → Intermediate)

Estimated AWS Cost: $15–$30

Module 4 — IAM & Security Foundation

#	Exercise	What You'll Learn
1	Create IAM users: `dev-user`, `ci-user`, `admin-user` with least-privilege policies	IAM users, policies
2	Create IAM roles: `ECSTaskRole`, `EC2-SSM-Role`, `JenkinsRole`	IAM roles, trust policies
3	Enable MFA on root, set password policy	Account security
4	Store DB passwords in AWS Secrets Manager, reference from services	Secrets management
5	Create a custom IAM policy that allows only `product-service` to access its specific DB	Fine-grained access
6	Enable CloudTrail, review API call logs	Auditing

Module 5 — Networking (VPC)

#	Exercise	What You'll Learn
1	Create a custom VPC: `10.0.0.0/16`	VPC fundamentals
2	Create 2 public + 2 private subnets across 2 AZs	Subnet design, AZs
3	Set up Internet Gateway + NAT Gateway	Routing, internet access
4	Create route tables: public → IGW, private → NAT	Route tables
5	Create Security Groups: `sg-frontend`, `sg-backend`, `sg-database`	Layered security
6	Test: frontend SG allows 80/443, backend only from frontend SG, DB only from backend SG	SG chaining

Module 6 — ECR (Container Registry)

#	Exercise	What You'll Learn

1	Create ECR repos: `devops-shop/product-service`, `devops-shop/order-service`, etc.	ECR basics
2	Authenticate Docker to ECR, push all 5 images	`aws ecr get-login-password`
3	Enable image scanning on push	Vulnerability scanning
4	Set lifecycle policies: keep only last 10 images	Cost optimization
5	Set up cross-account pull (simulate with a second IAM role)	ECR policies

Module 7 — RDS (Databases)

#	Exercise	What You'll Learn
1	Create 2 RDS MySQL instances: `product-db` and `order-db` (db.t3.micro)	RDS setup
2	Place in private subnets, accessible only from backend SG	DB subnet groups
3	Update service `application.yml` to use RDS endpoints	App configuration
4	Enable automated backups (7-day retention)	Backup strategy
5	Create a read replica for product-db	Read scaling
6	Test failover by rebooting primary	High availability

Module 8 — S3 & CloudFront

#	Exercise	What You'll Learn
1	Build React frontend (`npm run build`), deploy to S3 static hosting	S3 static hosting
2	Create CloudFront distribution pointing to S3	CDN, edge caching
3	Configure CloudFront → API Gateway origin for `/api/*` routes	Origin routing
4	Enable S3 versioning + lifecycle rules (move old to Glacier after 30d)	Storage classes
5	Block public access, use OAC (Origin Access Control) for CloudFront only	S3 security

---

Phase 3: Container Orchestration (Intermediate)

Estimated AWS Cost: $30–$50

Module 9 — ECS with Fargate

#	Exercise	What You'll Learn
1	Create an ECS Cluster (`devops-shop-cluster`)	ECS fundamentals
2	Write Task Definitions for all 5 services (Fargate, 0.25 vCPU, 512MB)	Task definitions, container defs

3	Create an ALB with target groups: `/api/products/**` → product-svc, `/api/orders/**` → order-svc	ALB path-based routing
4	Create ECS Services with desired count = 2, attach to ALB	Service deployment
5	Configure Service Discovery (Cloud Map) — replace Eureka with AWS-native	Service mesh basics
6	Set up auto-scaling: scale on CPU > 70%, min 1, max 4	ECS auto-scaling
7	Perform rolling update: change a product-service response, push new image, update service	Zero-downtime deployment
8	Set up ECS Exec for debugging running containers	Container debugging

Module 10 — EKS (Kubernetes)

#	Exercise	What You'll Learn
1	Create EKS cluster with `eksctl` (2x t3.medium nodes)	EKS setup
2	Write Kubernetes manifests: Deployment + Service for each microservice	K8s core objects
3	Create ConfigMaps for `application.yml` configs, Secrets for DB passwords	Config management
4	Deploy Ingress Controller (AWS ALB Ingress), configure path-based routing	Ingress
5	Set up HPA: auto-scale product-service on CPU > 50%	Horizontal Pod Autoscaler
6	Create PersistentVolumeClaims backed by EBS for any stateful needs	Storage
7	Implement liveness/readiness probes using Spring Actuator endpoints	Health checks
8	Perform Canary deployment: route 10% traffic to v2 of order-service	Deployment strategies
9	Set up Namespace isolation: `dev`, `staging`, `prod`	Multi-environment
10	Implement NetworkPolicies: order-service can call product-service, not vice versa	Network security
11	Use Kustomize or Helm charts to template all manifests	Package management

Cost Tip: Delete EKS cluster when not practicing. Nodes cost ~$1.70/day.

Phase 4: Infrastructure as Code (Intermediate → Advanced)

Estimated AWS Cost: $5–$15 (reuses above infra)

Module 11 — Terraform

#	Exercise	What You'll Learn
1	Install Terraform, create `infra/` directory, write VPC module	Terraform basics, HCL

2	Create modules: `modules/vpc`, `modules/ecs`, `modules/rds`, `modules/ecr`	Module design
3	Use `terraform.tfvars` for dev vs prod environments	Variables, workspaces
4	Store state in S3 + DynamoDB locking	Remote state
5	Write complete ECS Fargate stack in Terraform (VPC → ALB → ECS → RDS)	Full IaC deployment
6	Write complete EKS stack in Terraform (VPC → EKS → Node Groups → Add-ons)	EKS via Terraform
7	Add `terraform plan` output to PR comments	Plan review
8	Implement `terraform destroy` for cost savings (tear down nightly)	Lifecycle management
9	Use `data` sources to reference existing AWS resources	Data sources
10	Write Terraform tests with `terraform test` or Terratest	IaC testing

Sample Terraform Directory Structure:

```
infra/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       └── terraform.tfvars
├── modules/
│   ├── vpc/
│   ├── ecs/
│   ├── eks/
│   ├── rds/
│   ├── ecr/
│   └── alb/
└── backend.tf
```

Phase 5: CI/CD with Jenkins (Intermediate → Advanced)

Estimated AWS Cost: $10–$20

Module 12 — Jenkins Setup & Pipelines

#	Exercise	What You'll Learn
1	Deploy Jenkins on EC2 (t3.medium) with Docker	Jenkins installation
2	Install plugins: Docker Pipeline, AWS Steps, Git, Blue Ocean	Plugin management

3	Create a `Jenkinsfile` for product-service: Build → Test → Docker Build → Push to ECR	Declarative pipelines
4	Add stages: Lint, Unit Test, Integration Test, SonarQube scan	Quality gates
5	Create a multi-branch pipeline for all services	Monorepo CI
6	Add `Jenkinsfile` for Terraform: Plan → Manual Approve → Apply	IaC pipelines
7	Implement Jenkins shared libraries for common steps	Pipeline reuse
8	Deploy to ECS: update task definition + service in the pipeline	CD to ECS
9	Deploy to EKS: `kubectl apply` or Helm upgrade in pipeline	CD to EKS
10	Set up webhooks: GitHub push → auto-trigger Jenkins build	Event-driven CI
11	Add Slack/email notifications on build failure	Alerting
12	Implement Blue-Green deployment pipeline for zero-downtime releases	Advanced deployment

Sample Jenkinsfile (product-service):

```
pipeline {
    agent any
    environment {
        AWS_ACCOUNT_ID = credentials('aws-account-id')
        ECR_REPO = "${AWS_ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/devops-shop/product-
service"
    }
    stages {
        stage('Checkout')  { steps { checkout scm } }
        stage('Test')      { steps { dir('product-service') { sh 'mvn test' } } }
        stage('Build Jar') { steps { dir('product-service') { sh 'mvn package -DskipTests' }
} }
        stage('Docker Build & Push') {
            steps {
                sh "docker build -t ${ECR_REPO}:${BUILD_NUMBER} product-service/"
                sh "aws ecr get-login-password | docker login --username AWS --password-
stdin ${ECR_REPO}"
                sh "docker push ${ECR_REPO}:${BUILD_NUMBER}"
            }
        }
        stage('Deploy to ECS') {
            steps {
                sh """
                    aws ecs update-service --cluster devops-shop \
                        --service product-service \
                        --force-new-deployment
                """
            }
        }
```

```
    }
}
```

Phase 6: Observability (Advanced)

Estimated AWS Cost: $10–$25

Module 13 — Logging

#	Exercise	What You'll Learn
1	Configure Spring Boot services to output JSON logs (Logback + logstash-encoder)	Structured logging
2	Send ECS/EKS logs to CloudWatch Logs using `awslogs` driver or Fluent Bit	Centralized logging
3	Create CloudWatch Log Insights queries: error rate, slowest endpoints, trace by orderId	Log analysis
4	Set up ELK stack (Elasticsearch + Logstash + Kibana) on EC2 or use OpenSearch	Advanced log mgmt
5	Create log-based alarms: alert if ERROR count > 10 in 5 min	Alert-based logging

Module 14 — Metrics & Monitoring

#	Exercise	What You'll Learn
1	Add Spring Actuator + Micrometer Prometheus endpoint to all services	App metrics
2	Deploy Prometheus on EKS (helm install prometheus)	Metrics collection
3	Deploy Grafana on EKS, connect to Prometheus	Dashboarding
4	Build Grafana dashboards: JVM memory, HTTP request rate, DB connection pool, order throughput	Custom dashboards
5	Set up CloudWatch Container Insights for ECS/EKS	AWS-native monitoring
6	Create CloudWatch alarms: CPU > 80%, 5xx > 1%, unhealthy targets	Alerting
7	Create a composite CloudWatch dashboard for the entire application	Operational view

Module 15 — Distributed Tracing

#	Exercise	What You'll Learn
1	Add OpenTelemetry SDK to all Spring Boot services	Instrumentation
2	Deploy Jaeger or AWS X-Ray as tracing backend	Trace collection

3	Trace a full order flow: frontend → gateway → order-service → product-service	End-to-end tracing
4	Identify bottleneck: add artificial delay to product-service, find it via traces	Performance debugging
5	Create X-Ray service map for the entire application	Service topology

---

Phase 7: Advanced AWS & Production Readiness (Advanced → Expert)

Estimated AWS Cost: $20–$40

Module 16 — Route 53 & SSL

#	Exercise	What You'll Learn
1	Register a free domain or use a subdomain with Route 53	DNS management
2	Create ACM certificate for `*.yourdomain.com`	SSL/TLS
3	Attach cert to ALB, force HTTPS redirect	HTTPS termination
4	Create DNS records: `app.yourdomain.com` → CloudFront, `api.yourdomain.com` → ALB	DNS routing
5	Set up weighted routing: 90% to v1, 10% to v2	Traffic management

Module 17 — Parameter Store & Secrets

#	Exercise	What You'll Learn
1	Migrate all DB URLs, credentials to SSM Parameter Store	Externalized config
2	Use Secrets Manager for rotation-capable secrets	Secret rotation
3	Update Terraform to create parameters, reference in ECS task defs	IaC + config
4	Implement in EKS using External Secrets Operator	K8s secret sync

Module 18 — Cost Optimization

#	Exercise	What You'll Learn
1	Enable AWS Cost Explorer, set up a $200 budget alarm	Cost tracking
2	Use Spot instances for EKS worker nodes (save 60%)	Spot strategy
3	Schedule ECS desired count = 0 at night (Lambda + EventBridge)	Scheduling
4	Right-size RDS: use db.t3.micro, stop when not used	RDS cost mgmt
5	Review with AWS Trusted Advisor	Recommendations

Module 19 — Security Hardening

#	Exercise	What You'll Learn
1	Enable GuardDuty for threat detection	Threat detection
2	Run AWS Config rules: ensure encryption, public access blocks	Compliance
3	Scan Docker images with ECR scanning + Trivy in Jenkins	Supply chain security
4	Implement WAF rules on ALB: rate limiting, SQL injection blocking	WAF
5	Enable VPC Flow Logs, analyze with Athena	Network forensics
6	Run `kube-bench` on EKS for CIS benchmark compliance	K8s security

Phase 8: Expert-Level Projects (Expert)

Estimated AWS Cost: $15–$30

Module 20 — Full GitOps with ArgoCD

#	Exercise	What You'll Learn
1	Install ArgoCD on EKS	GitOps setup
2	Create a `k8s-manifests` repo, connect to ArgoCD	Declarative CD
3	Push a change → ArgoCD auto-syncs to EKS	Automated deployment
4	Implement progressive delivery with Argo Rollouts	Canary/blue-green

Module 21 — Service Mesh (Istio / App Mesh)

#	Exercise	What You'll Learn
1	Install Istio on EKS	Service mesh
2	Enable mTLS between all services	Zero-trust networking
3	Configure traffic splitting: 90/10 between versions	Traffic management
4	Inject faults: 500 errors on 10% of product-service calls	Chaos engineering

Module 22 — Multi-Environment Pipeline (End-to-End)

Build the complete pipeline:

```
Developer pushes code
    → GitHub webhook triggers Jenkins
    → Jenkins: build, test, scan, push to ECR
    → Jenkins: terraform plan (dev) → apply
    → Jenkins: deploy to ECS/EKS dev
    → Run integration tests
    → Manual approval gate
    → Jenkins: terraform plan (prod) → apply
```

```
    → Jenkins: deploy to ECS/EKS prod (blue-green)
    → Smoke tests
    → Prometheus alerts if errors > threshold → auto-rollback
```

Module 23 — Disaster Recovery

#	Exercise	What You'll Learn
1	Set up RDS cross-region read replica	DR setup
2	Create AMI backups with AWS Backup	Backup automation
3	Write a runbook: failover to secondary region	DR documentation
4	Simulate failure: delete an ECS service, measure recovery time	RTO/RPO testing

---

Budget Planner ($200 Credits)

Resource	Est. Monthly Cost	Tip
EKS Control Plane	$73/mo	Delete when not using
EKS Nodes (2x t3.medium)	$50/mo	Use Spot, stop at night
ECS Fargate (5 services, 0.25vCPU)	$15/mo	Scale to 0 at night
RDS (2x db.t3.micro)	$15/mo	Stop when not practicing
NAT Gateway	$32/mo	Biggest hidden cost! Delete when done
ALB	$16/mo	—
EC2 Jenkins (t3.medium)	$25/mo	Stop when not using
CloudWatch / X-Ray	$5–10/mo	—
ECR Storage	$1/mo	—
S3 + CloudFront	$1/mo	—

Strategy to stay under $200:

1. **Never run EKS + ECS simultaneously** — practice ECS first (Phase 3), destroy, then practice EKS
2. **Tear down every night** — use `terraform destroy` or stop resources
3. **Delete NAT Gateway** when not in use (costs $1/day doing nothing)
4. **Set a CloudWatch billing alarm at $50, $100, $150**
5. Practice Phases 1–2 locally (free), then go to AWS for Phases 3+
6. Each module takes 1–3 days. Plan 6–8 weeks total

Recommended Learning Order

```
Week 1:    Module 1 (Git) + Module 2 (Docker)                    — FREE
Week 2:    Module 3 (Linux) + Module 4 (IAM)                     — ~$2
```

```
Week 3:    Module 5 (VPC) + Module 6 (ECR) + Module 7 (RDS)      — ~$10
Week 4:    Module 9 (ECS Fargate) + Module 8 (S3/CF)             — ~$20
Week 5:    Module 11 (Terraform: ECS stack)                       — ~$15
Week 6:    Module 12 (Jenkins CI/CD)                              — ~$25
Week 7:    Module 10 (EKS) + Module 11 (Terraform: EKS)          — ~$40
Week 8:    Module 13-15 (Observability)                           — ~$15
Week 9-10: Modules 16-23 (Advanced/Expert)                        — ~$30
```

Total estimated: ~$160 with headroom for mistakes and retries.

---

23 modules | 150+ hands-on exercises | Beginner to Expert